FULL TECHNICAL REPORT: AAST Website Scraping & Structural Problems

Introduction During the development of the AAST Knowledge Graph & RAG System, extensive crawling and scraping were performed on the official AAST website. Throughout this process, we discovered that the website suffers from major structural, architectural, and API-related issues that significantly complicate automated data extraction.

This report summarizes all core problems, categorized and explained, with real technical impact.

1) Inconsistent Website Architecture The AAST website is not a single unified system. Instead, it is a collection of 7+ independent systems stitched together: - contenttemp.php pages (legacy CMS) - programtemp.php pages (different CMS) - Staff/CV pages (cv.php) - Presidency website (different system) - Training centers (Vue/React-based pages) - Static HTML templates - Dynamic menus (JavaScript-heavy navigation) - AJAX APIs that load content after page render - Old Joomla/WordPress remnants

Impact: - No standard structure - No unified schema or data model - Every college has its own mini-website - Crawlers must handle each subsystem differently

2) Dynamic Navigation – Zero Real HTML Links Most navigation menus do NOT exist in the static HTML. Links only appear after hover, dropdown interaction, JavaScript execution, or AJAX requests. HTML crawlers see almost no links at all.

Impact: - Deep discovery impossible from homepage - Need manual seeding of 5600+ URLs - JS-dependent interactions required for every visit

3) Broken, Inconsistent & Unstable APIs APIs return inconsistent structures, empty objects, NULL, or unexpected redirects: - getPageContent2021.php?page_id=XXXX - getProgramData2021.php?program_id=XXX - getCvNew.php?ser=XXXX

Impact: - Impossible to rely on API responses - Need fallback HTML extraction - Multi-layer error handling - Many IDs fail or redirect

4) Extremely Noisy HTML Pages contain excessive JS files, analytics scripts, duplicated headers, and inline CSS/JS.

Impact: - DOM is large and slow - Heavy parsing required - Playwright slows down - Text extraction needs deep filtering

5) Content Not Available Without JavaScript Most content loads via AJAX, making static requests useless.

Impact: - Normal crawlers = useless - Requires JS execution - Heavy resource consumption

6) Redirect Hell Many URLs behave unpredictably, including loops, forced homepage redirects, and new-tab navigation.

Impact: - Breaks crawl chains - Requires redirect logic - Causes missing/duplicated nodes

7) Lack of Canonical URL Structure Identifiers change across templates: program_id vs id, unit_id vs unit_item.

Impact: - No predictable patterns - Hard to build relationships - Require brute-force exploration

8) Duplicate Pages Everywhere English, Arabic, print, template, legacy, backup versions all exist.

Impact: - 4–7 duplicates per page - Heavy deduplication required

9) File Attachments Without Metadata PDFs lack names, descriptions, dates, or metadata.

Impact: - Hard to categorize automatically - Need manual mapping rules

10) Every College Uses Its Own System Different colleges rely on different engines and layouts.

Impact: - No unified navigation - No uniform schema

11) Chaotic Content IDs page_id and program_id have no logical structure.

Impact: - Hard to infer relations - Requires brute-force discovery

12) Hidden Content in JavaScript Roadmaps, grading systems, publications load via hidden AJAX endpoints.

13) Heavy Headers Loaded 2–3 Times Per Page Duplicate menus and scripts inflate DOM size.

14) Multilingual System Without Proper URL Separation Language switching uses redirects instead of clean URLs.

Impact: - Crawlers get stuck - Pages duplicated under different URLs

Final Conclusion AAST website is not scraping-friendly. It is inconsistent, unstable, JS-dependent, API-unreliable, and lacks a unified structure. Hybrid scraping with JS rendering, API fallbacks, and manual seeding is required.